

Research Advisor Briefing: Neural SC Decoder for MAC Polar Codes

What I need from you

I need strategic advice on how to spend my time, not technical suggestions like “use this loss function” or “try this architecture.” I have an AI coding assistant that handles implementation. What I need is a coach/mentor perspective:

- Am I chasing the right problem?
- Should I pivot or double down?
- What’s the most efficient path to a publishable result?
- Am I falling into common research traps?

The project in one paragraph

I’m building a neural network that replaces the analytical successive cancellation (SC) decoder for polar codes on a two-user Multiple Access Channel (MAC). The neural decoder should match the analytical SC’s block error rate (BLER) while also working on channels where analytical decoding is impossible (channels with memory, unknown channels). The project has been running for 8 sessions (~200+ CPU-hours of training) across March-April 2026.

What works (proven results)

The **sequential training approach** works. A neural decoder trained with $O(N \log N)$ sequential gradient depth matches or beats analytical SC:

Block length N	Neural BLER	SC BLER	Ratio
32	0.037	0.046	0.80x (beats SC)
64	0.020	0.025	0.80x (beats SC)
128	0.0185	0.016	1.16x (close, training interrupted)

This uses a 153K-parameter model ($d=32$ embeddings) with curriculum learning (train at $N=16$, transfer weights to $N=32$, then $N=64$, then $N=128$). Each stage takes 5-30 hours on Apple M-series CPU.

Additionally: - CRC-aided neural SCL: zero errors at $N=128$ with list size 8 (beats analytical SCL)
- ISI-MAC (channel with memory): 19% improvement over memoryless SC baseline - BEMAC (discrete channel): neural decoder beats SC by 2x at $N=256$

The scaling wall

At $N=256$, sequential training collapses. The gradient must flow through ~1500 sequential MLP operations. Training takes ~1.5 seconds per iteration and doesn’t converge even after days of training.

This is the central problem: **sequential training works but doesn’t scale past $N=128$.**

The parallel training attempt (fast_ce)

To bypass the gradient depth problem, I tried “fast cross-entropy” (fast_ce) — a parallel training method from the single-user NPD literature. Instead of walking the tree sequentially, it processes all positions at each tree depth in parallel, reducing gradient depth from $O(N \log N)$ to $O(\log N)$.

For single-user polar codes, fast_ce works perfectly. The key reason: the BitNode operation has a sign-flip symmetry. A wrong binary prediction just negates the embedding — a transformation the model has seen equally often during training. So the model is inherently robust to its own errors.

For the 4-class MAC, fast_ce initially appeared to fail completely (BLER=1.0 on sequential decode). I spent significant time (multiple sessions) trying to fix this: - WHT decomposition to diagonalize CalcLeft - Noisy teacher forcing - Scheduled sampling - PSS (Parallel Scheduled Sampling) - Various loss formulations

All gave BLER=1.0 on sequential decode.

The breakthrough (today)

I discovered the sequential decoder had a bug. The fast_ce parallel pass and the sequential decoder used incompatible domains:

- fast_ce trains with **codeword-domain** values and **bit-reversed** leaf targets
- The sequential decoder was feeding **message-domain** values in **natural order**

These are related by the polar encoding transform + bit-reversal permutation. The model was trained on one representation but evaluated on a completely different one.

After fixing the bug (correct bit-reversal mapping + codeword-domain butterfly reconstruction in the decoder), fast_ce immediately achieves **BLER=0.37 (8x SC)** — down from BLER=1.0. This matches earlier theoretical predictions and confirms the model IS learning the right tree operations.

Current status (right now)

I’m testing 6 ideas in parallel to close the remaining 8x gap between fast_ce (0.37) and SC (0.046) at $N=32$:

1. **Hybrid: fast_ce pretrain -> sequential fine-tune** — Use fast_ce for cheap initialization, then a few sequential iterations to close the exposure bias gap
2. **PSS (Parallel Scheduled Sampling)** — Expose model to its own errors during parallel training
3. **More training / larger model (d=32)** — Maybe 15K iters or $d=16$ is insufficient
4. **Scale to $N=64$** — Check if the 8x gap is stable or grows
5. **All-depths vs leaf-only loss** — Different loss placement
6. **Group-equivariant MLPs** — Architectural symmetry constraint

Results are pending (~1-2 hours).

The bigger picture

What’s at stake

- **If hybrid (fast_ce -> sequential fine-tune) works:** This solves the scaling problem. Fast_ce gives a cheap $O(\log N)$ initialization at any N , then a small number of sequential

iterations closes the gap. Training at $N=256$ or $N=1024$ becomes feasible.

- **If fast_ce alone can reach ~2x SC:** That’s already a publishable result — first working parallel-trained neural MAC decoder.
- **If nothing closes the gap:** I fall back to the sequential approach, which works at $N \leq 128$ and might reach $N=256$ with enough patience (3-4 days of training).

Paper readiness

All materials are prepared: 8 figures, 60+ paper literature survey, theoretical analysis, paper outline. The main missing piece is a strong result at $N=256$.

Time constraints

This is for an academic paper. I have limited compute (Apple M-series CPU, no GPU — MPS is 5x slower for this workload). Each training run at $N=128$ takes ~28 hours, at $N=256$ would take 3-4 days.

What I’ve learned about this problem

1. **The 4-class MAC is fundamentally harder than single-user.** The sign-flip symmetry that makes single-user NPD work extends to 4-class ($Z_2 \times Z_2$ group), but the error patterns compound differently.
2. **Bugs can hide for weeks.** The domain mismatch between fast_ce and sequential decode was present in ALL fast_ce experiments. Every “failed approach” (WHT, PSS, noisy teacher forcing) was tested on a broken decoder. Some of these might actually work now.
3. **Sequential training works but is slow.** The $d=32$ model beats SC at $N \leq 64$ and was converging at $N=128$ before training died. Patience + compute might be enough.
4. **The z-encoder bottleneck is real.** Compressing continuous channel output to $d=16$ dimensions loses ~0.04 bits/symbol. At $N=256$, this accumulates to ~10 bits of lost information. $d=32$ helps significantly.

Questions for the advisor

1. Given the bug fix breakthrough, should I re-test ALL the previously “failed” approaches (WHT, scheduled sampling, etc.) or focus on the most promising new direction (hybrid)?
2. Is pursuing fast_ce the right strategy at all? The sequential approach already works at $N \leq 128$. Maybe I should just be patient and train $d=32$ at $N=256$ for 4 days.
3. For the paper, what’s the minimum result I need at $N=256$? Is “matches SC at $N \leq 128$, degrades gracefully at $N=256$ ” publishable? Or do I need to match SC at $N=256$?
4. I’m spending a lot of time on training methodology (fast_ce, PSS, curriculum). Should I instead be spending time on architecture (equivariant networks, attention, larger models)?
5. Am I over-optimizing at $N=32$? The real target is $N=256+$. Should I skip $N=32$ experiments and go straight to larger N ?

6. The CRC-aided SCL result (zero errors at $N=128$) and the ISI-MAC result (19% improvement) are already strong. Should I write the paper around those results and treat the scaling problem as future work?